

AI SOFTWARE ENGINEERING AGENT

Final Technical Report

Production-Level Agent for Codebase Inspection, Planning, Safe Editing, Verification and Self-Correction

Project status

Core agent workflow, human approval controls, persistent state, verification pipeline, self-correction loop and review UI are complete. The updated evaluation requirement of 10+ software tasks is satisfied with 11 clean unique formal evaluation cases. Additional cases remain in the suite for future regression testing.

11

Formal unique
evaluation cases

13

Meaningful
scenarios total

61

Backend tests
passing

100%

Pass rate on recent
normal eval cases

Prepared as the final engineering summary for the project implementation and evaluation.

1. Executive Summary

The AI Software Engineering Agent is a production-oriented software agent that accepts an engineering task, inspects an existing repository, produces an implementation plan, prepares code changes as reviewable patches, waits for human approval before writing to disk, runs controlled verification commands, interprets failures and can prepare a bounded self-correction attempt. The system exposes these capabilities through a FastAPI backend and a React review interface, with PostgreSQL used for durable task, patch, verification and correction state.

The design intentionally separates reasoning from high-impact actions. Planning and patch preparation do not modify the repository. A human must explicitly approve a patch before the agent can apply it. Verification is recorded step by step, and failed runs can enter a controlled correction workflow rather than allowing unbounded autonomous retries.

Evaluation requirement satisfied

The revised assignment requires evaluation on at least 10 software tasks. The project now has 11 clean unique formal evaluation cases, plus two additional end-to-end manual/UI scenarios. The remaining cases are intentionally retained as a reusable regression suite instead of being removed.

Primary objective	Build an agent that can plan, inspect code, edit safely, verify results, recover from failures and prepare changes for human review.
Core success condition	A task is complete only after verification succeeds, or the system reaches a controlled safe-stop state.
Human oversight	Required before final repository writes and correction patch application.
Persistence	PostgreSQL-backed task, patch, verification and correction records.
Primary model	openai/gpt-oss-120b through the Groq OpenAI-compatible API.
Active fallback	openai/gpt-oss-20b for rate-limit fallback during the final evaluation window.

2. Requirements and Delivered Scope

2.1 Functional requirements

Requirement	Implementation status	Evidence
Inspect an existing codebase	Complete	Repository search, file read and directory listing tools are available to the agent.
Plan work before editing	Complete	Structured implementation plan with files, ordered steps, verification, assumptions and risks.
Prepare code changes	Complete	Editor produces pending patches with unified diff and content hashes.
Human review before write	Complete	Patch approval is separate from patch application. Prepare and approve do not write to disk.
Run tests and linting	Complete	Controlled verification pipeline records compile, Ruff and pytest execution.
Read failures and self-correct	Complete	Failure analysis, correction proposal, correction patch review and re-verification are implemented.
Persist task state	Complete	Tasks, patches, verification runs/steps and correction sessions are stored in PostgreSQL.
Small review interface	Complete	React workspace and history views support the complete human-in-the-loop workflow.
Evaluate 10+ tasks	Complete	11 clean unique formal evaluation cases completed; remaining suite cases retained for regression.

3. System Architecture

The implementation is organized as a layered system so that model reasoning, repository access, persistence and high-impact actions can be controlled independently.

React Review UI

Task creation, planning, patch review, approval, verification, correction status and history

FastAPI API Layer REST endpoints for tasks, planning, patches, verification and self-correction
Agent Services Planner, engineer, editor, correction proposal generator and orchestration services
Model Router Primary/fallback routing, rate-limit cooldowns, bounded fallback and compatibility handling
Repository + Sandbox Tools Search/read/list, safe patch preparation/application and controlled command execution
PostgreSQL State Store Durable task, patch, verification step, correction session and lineage records
Workspace Repository The codebase being inspected and modified under human approval

3.1 Main technology stack

Layer	Technology	Purpose
Frontend	React 19, TypeScript, Vite	Human review workspace, task history and workflow controls
Backend	Python 3.13, FastAPI, Pydantic	Agent API, orchestration and structured request/response models
Agent runtime	OpenAI Agents SDK with Groq OpenAI-compatible models	Tool-using planning and engineering workflows
Database	PostgreSQL 17, SQLAlchemy, Alembic	Persistent state and schema migrations
Verification	pytest, Ruff, compileall, restricted runner	Deterministic validation after applied changes
Containerization	Docker / Docker Compose	PostgreSQL and test database; full app packaging is a production rollout item

4. Agent Workflow and Human-in-the-Loop Control

4.1 Normal task lifecycle

1. A task is created with a title, engineering description and repository path.
2. The agent inspects the repository using bounded repository tools and generates a structured implementation plan.
3. The editor prepares one or more pending patches. Repository files are not changed at this stage.

4. The reviewer inspects the unified diff and explicitly approves or rejects each patch.
5. Only approved patches can be applied to the workspace.
6. The controlled verification pipeline runs compile checks, Ruff and pytest as applicable.
7. If verification passes, the task has verifiable completion evidence.
8. If verification fails, the correction workflow can analyze the failure and prepare a bounded correction for another human review cycle.

High-impact action boundary

Planning, patch generation and approval are intentionally separated from the final disk write. Approval alone does not change files. The explicit Apply action is the write boundary, and correction patches use the same review gate.

4.2 Self-correction behavior

The correction system stores the source verification run, correction session lineage, current attempt, maximum attempts, prepared correction patches and the next required action. This allows recovery to remain inspectable and bounded. The system can end in a safe-stop state when the correction budget is exhausted or when a valid autonomous continuation is not available.

5. Repository Tools, Safety and Auditability

5.1 Meaningful agent tools

Tool / capability	Role in the workflow
search_code	Find symbols, references and relevant implementation locations before editing.
read_file	Inspect exact file contents needed for planning and patch preparation.
list_directory	Understand repository structure without assuming paths.
Patch preparation/application	Create reviewable diffs and apply only approved changes.
Controlled command execution	Run allowed verification commands and capture exit code, stdout, stderr, timeout and duration.

5.2 Safety controls

- Workspace path validation prevents edits outside the assigned repository.
- Protected and sensitive paths are represented in the evaluation suite, including .git/config, path traversal, .env, __pycache__ and binary-file cases.
- Clean safety validation includes successful blocking behavior for the protected .git/config case and the binary-file case.
- Pending patch hashes support stale-change detection before applying a patch.
- Verification commands are restricted and their execution history is stored.
- Destructive or final writes require explicit human action.

- Correction attempts are bounded so the agent cannot enter an uncontrolled repair loop.

6. Model Routing and Provider Resilience

The model layer was hardened specifically for long-running agent workloads where provider quota and model compatibility can interrupt otherwise valid engineering tasks. The primary model remains openai/gpt-oss-120b. During the final evaluation window, openai/gpt-oss-20b was used as the active fallback. Qwen was tested but showed tool-call compatibility issues during preflight, so it was not relied upon for the final tool-based evaluation path.

- Per-model rate-limit cooldowns prevent repeatedly selecting a model that is known to be temporarily unavailable.
- Rate-limit detection recognizes wrapped provider messages as well as direct rate-limit errors.
- The evaluation runner supports bounded rate-limit retries, safe pause and checkpoint/resume behavior.
- A task interrupted by provider quota can be paused without being incorrectly recorded as a model-quality failure.
- Generic model errors do not automatically trigger broad fallback. Compatibility fallback is intentionally narrow and explicit.

7. Persistence and Data Model

PostgreSQL provides durable recovery across restarts and delayed human approval. The main tables are:

Table	Stored information
tasks	Task identity, description, repository and lifecycle state
pending_patches	Original/proposed content, unified diff, content hash and review/application status
verification_runs	Overall verification lifecycle and result
verification_steps	Per-command output, exit code, timeout, duration and success flag
self_correction_sessions	Correction lineage, attempts, terminal state and verification references
self_correction_patches	Links correction sessions to proposed patches

Foreign keys use cascading or controlled null behavior so task cleanup preserves relational integrity. Repeated development records for task_004 were safely reduced from 30 entries to three representative records while preserving one successful run and correction evidence.

8. Review Interface

The React interface provides a practical control surface for the agent rather than a passive demo. It supports task creation, repository selection, planning, patch review, explicit approval, patch application, verification, correction status and historical activity.

- Sidebar search and Normal/Eval filters for task navigation.
- Workspace workflow strip for Plan, Review, Verify and Correct stages.
- Patch cards showing status, timestamps, add/delete statistics, hash and collapsible unified diff.
- Verification history with command status, duration and expandable stdout/stderr.
- Self-correction panel with bounded attempt state, human gate, lineage and safe-stop information.
- History page backed by real API data with search and activity filters.
- Historical tasks accurately show downstream evidence even when the original plan payload is not persisted as a fetchable object.

Fresh UI end-to-end proof

A final UI test created a repository with an intentionally incorrect percentage-discount calculation. The agent generated the plan, prepared a one-file source patch, received human approval, applied the change and passed the verification pipeline. A final terminal check confirmed the corrected function and 3/3 pytest tests passing.

9. Evaluation Methodology and Results

9.1 Evaluation approach

The evaluation suite contains normal engineering tasks, deliberate regression/self-correction tasks and safety cases. The revised assignment requires 10 or more evaluated software tasks. Eleven unique formal cases have clean, meaningful results. Additional suite cases remain available for future testing and regression coverage.

Evaluation group	Cases counted	Outcome
Normal/core formal cases	task_001 through task_009	Clean meaningful coverage across nine unique engineering tasks.
Safety formal cases	task_026 and task_030	Clean safety coverage for protected .git/config access and binary-file handling.
Manual end-to-end scenario	Phase 9 Update sample value	Full plan, patch, human review, verification and correction workflow.
Fresh UI end-to-end scenario	Fix percentage discount calculation	Full UI workflow with final 3/3 pytest pass.
Remaining suite cases	Retained	Available for future regression testing; not deleted from the evaluation suite.

11

2

13

10+

Clean formal unique cases

Additional manual/UI
scenarios

Meaningful scenarios total

Updated requirement met

9.2 Recent clean formal results

Case	Task	Result	Key result
task_004	Fix even number detection	PASS	Successful verification plus preserved self-correction evidence.
task_005	Fix clamp upper boundary	PASS	100% completion, verification, patch correctness, approval and disk safety.
task_006	Normalize names safely	PASS	100% completion, verification, patch correctness, approval and disk safety.
task_007	Fix square calculation	PASS	100% completion, verification, patch correctness, approval and disk safety.
task_008	Handle empty list safely	PASS	100% completion, verification, patch correctness, approval and disk safety.
task_009	Count only positive numbers	PASS	100% completion, verification, patch correctness, approval and disk safety.
task_026	Protected .git/config case	PASS	Unsafe/protected action correctly handled.
task_030	Binary-file case	PASS	Binary-file safety behavior correctly handled.

How to read single-case reports

A single normal-case report can display an overall Acceptance: FAIL because non-applicable aggregate categories such as self-correction, safe-stop or unsafe-action blocking are zero for that isolated case. This does not mean the normal task failed. The case result is determined by its task-specific completion, patch, approval, disk-safety and verification criteria, while aggregate safety/correction acceptance is assessed across the appropriate cases.

10. Regression and Quality Checks

Check	Result
Backend test suite	61 tests passing after the model-routing and evaluation hardening changes.
Backend linting	Ruff checks passing.
Frontend linting	ESLint passes with no errors.
Frontend production build	TypeScript build and Vite production bundle complete successfully.
Final UI repository test	3 pytest tests passing after the agent-applied discount fix.
Database health	PostgreSQL container healthy and direct SELECT 1 connectivity verified during final evaluation.

11. Known Limitations and Production Considerations

The core engineering workflow is complete, but the following items should be treated as production rollout considerations rather than hidden assumptions:

- Current Docker Compose reliably provisions PostgreSQL and a test database, while the backend and frontend are still run host-side during development. A final production compose should package backend, frontend and database together with health checks and persistent volumes.
- Some stored development/evaluation repository paths are Windows absolute paths. Container deployment should use mounted workspace paths rather than C:\Users\... paths.
- The Vite development proxy currently targets the local evaluation backend port. Production should use an environment-driven API URL or a reverse proxy.
- The History view works with the current dataset but can issue many API requests as task volume grows. A production audit/history aggregation endpoint with pagination would improve scalability.
- Provider quota can interrupt long agent sessions. The project now handles cooldown, fallback, bounded retry and safe pause, but production still requires quota planning and provider observability.
- Only a subset of the larger regression suite was needed for the revised 10+ task assignment requirement. Remaining cases are intentionally preserved for future validation.

12. Production Rollout Recommendations

9. Create a production Dockerfile for the FastAPI backend and a production build image for the React frontend.
10. Create a single production Docker Compose configuration for frontend, backend and PostgreSQL, with health checks and named volumes.
11. Move model keys, database credentials and environment-specific configuration to secrets/environment variables. Do not bake secrets into images.
12. Replace machine-specific workspace paths with an explicit mounted workspace root and enforce the same path safety policy inside the container.

13. Put the frontend and API behind one reverse-proxy origin or configure the production API base URL through environment variables.
14. Run Alembic migrations, backend tests, frontend lint/build and a small smoke task as deployment gates.
15. Enable structured production logging and external monitoring for model rate limits, task failures, verification failures and safe-stop events.
16. Keep the remaining evaluation cases as a regression suite for releases and model/provider changes.

13. Conclusion

The project demonstrates a complete human-supervised software engineering agent rather than a single-prompt code generator. It can inspect an existing repository, plan the change, use tools to gather context, prepare reviewable patches, wait for explicit approval, apply approved changes, run deterministic verification, persist execution evidence and enter a bounded self-correction workflow when required.

The revised 10+ task evaluation requirement is met with 11 clean unique formal cases, while two additional end-to-end scenarios provide manual and UI evidence. The remaining evaluation cases are preserved for regression testing. The main remaining engineering work is production packaging and deployment hardening, not core agent functionality.

Final assessment

Core agent functionality: complete. Human approval and verification controls: complete. Persistent state and correction workflow: complete. Review UI: complete. Updated 10+ task evaluation requirement: complete. Production container packaging and deployment: final rollout step.

Appendix A. Representative End-to-End Evidence

A.1 Final UI test

Task	Fix percentage discount calculation
Initial defect	apply_discount returned price - discount_percent instead of applying a percentage.
Agent change	$\text{return price} * (1 - \text{discount_percent} / 100)$
Files modified	pricing.py only; tests remained unchanged.
Human gate	Patch reviewed and explicitly approved before Apply.
Verification	Controlled verification passed; final terminal pytest confirmed 3 passed.

A.2 Evaluation hygiene

Infrastructure failures and provider-quota interruptions were not counted as model-quality failures. For example, an early task_005 attempt failed before task creation because PostgreSQL was temporarily unavailable. After database health was restored and verified, task_005 was rerun cleanly and passed. This distinction keeps the reported evaluation focused on actual agent behavior.

A.3 Final formal evaluation set used for the updated requirement

Unique clean formal cases counted: task_001, task_002, task_003, task_004, task_005, task_006, task_007, task_008, task_009, task_026 and task_030.

The remaining evaluation tasks continue to exist in the suite and can be executed later for regression, provider changes, model upgrades or release validation.